

Modular periodicity of the horizontal-domino-residue constrained tiling count on the $4 \times n$ board (Lean 4 machine-checked claim of record)

Aurora0134*

September 30, 2026

Abstract

We record a machine-checked result about a pure integer sequence $a_4: \mathbb{N} \rightarrow \mathbb{Z}$ defined by initial values together with an order-twelve constant-coefficient recurrence: its residue modulo 2 depends only on $n \bmod 15$ and its residue modulo 3 depends only on $n \bmod 30$, with the two residue patterns exhibited explicitly. The combinatorial reading of a_4 is the number of domino tilings of a $4 \times n$ board counted by the number of horizontal dominoes, modulo 3; that identification is *not* proved here. All statements are verified by the Lean 4 kernel under the pinned mathlib revision, with no `sorry` and axioms confined to `{propext, Quot.sound, Classical.choice}`. Novelty: search found no occupying record; the strongest licensed claim is that no record of this constrained-selection object was found in the channels queried. This note is a priority claim of record, not a full paper.

1 Introduction

This note records a formalized result produced by an AI4Math automated pipeline, in which propositions and proofs are generated by a language model and adjudicated solely by the Lean 4 kernel. The result concerns a pure sequence-level object: an order-twelve constant-coefficient integer recurrence together with the modular periodicity of its residues. The note is a priority

*Contact: via the GitHub account Aurora0134; ORCID placeholder — to be filled in by the author before any upload. No personal information is carried in this note.

claim of record, deposited for timestamping; the corresponding narrative paper has been deliberately deferred by the author (see the card), so no full exposition is attempted here. Verification strength is stated in Section 4: strict kernel compilation, zero `sorry`, standard axiom whitelist. No literature survey is included—that expectation belongs to arXiv moderation and does not apply to a Zenodo deposit.

2 Statement

We first fix notation. Let $a_4: \mathbb{N} \rightarrow \mathbb{Z}$ be the sequence with initial values

$$a_4(0), \dots, a_4(11) = 1, 1, 1, 1, 10, 37, 110, 269, 701, 2000, 6020, 17495$$

extended by the order-twelve constant-coefficient recurrence

$$\begin{aligned} a_4(n) = & 3a_4(n-1) - 3a_4(n-2) + a_4(n-3) + 18a_4(n-4) - 9a_4(n-5) + 20a_4(n-6) \\ & + 15a_4(n-7) - 18a_4(n-8) + a_4(n-9) + 9a_4(n-10) - a_4(n-12), \end{aligned}$$

and let pat2_{15} and pat3_{30} be the two explicit residue patterns recorded below (fifteen and thirty entries respectively). Two facts are formalized, and both are main results; this track contains no definitional-unfolding lemma of the kind recorded in the sibling three-by- n track.

Theorem 1 (mod-2 residue determined by $n \bmod 15$; main result). *For all $n, r \in \mathbb{N}$, if $n \bmod 15 = r$ then $a_4(n) \bmod 2 = \text{pat2}_{15}(r)$.*

Theorem 2 (mod-3 residue determined by $n \bmod 30$; main result). *For all $n, r \in \mathbb{N}$, if $n \bmod 30 = r$ then $a_4(n) \bmod 3 = \text{pat3}_{30}(r)$.*

The reading of a_4 as a tiling count is fixed throughout as follows: $a_4(n)$ is the number of domino tilings of the $4 \times n$ board counted by the number of horizontal dominoes, modulo 3 — that is, the sum over those tilings whose horizontal-domino count lies in the residue class 0 modulo 3. This is deliberately *not* the tiling count taken modulo 3, which is a different object.

Listing 1: Frozen formal statement (Lean 4, verbatim from `formalized/04-mod3x4-statements.lean`, lines 26–106), with the two proof-body placeholder lines omitted (see the note below the listing). sha256 `b3970831...`).

```
1 /-- **T4 主序列  $a_4$  ( $order-12$  递推) **.
2 初值  $a_4(0..11) = 1, 1, 1, 1, 10, 37, 110, 269, 701, 2000, 6020,$ 
    $17495;$ 
```

```

3  递推  $a_4(n) = 3 \cdot a_4(-n1) - 3 \cdot a_4(-n2) + a_4(-n3) + 18 \cdot a_4(-n4) - 9 \cdot a_4(-n5) + 20 \cdot a_4(-n6)$ 
4       $+ 15 \cdot a_4(-n7) - 18 \cdot a_4(-n8) + a_4(-n9) + 9 \cdot a_4(-n10) - a_4(-n12)$ 。 -/
5  def a4 :  $\mathbb{N} \rightarrow \mathbb{Z}$ 
6      | 0 => 1
7      | 1 => 1
8      | 2 => 1
9      | 3 => 1
10     | 4 => 10
11     | 5 => 37
12     | 6 => 110
13     | 7 => 269
14     | 8 => 701
15     | 9 => 2000
16     | 10 => 6020
17     | 11 => 17495
18     | n + 12 =>
19         3 * a4 (n + 11) - 3 * a4 (n + 10) + a4 (n + 9) + 18 * a4 (n
20         + 8)
21         - 9 * a4 (n + 7) + 20 * a4 (n + 6) + 15 * a4 (n + 5) - 18
22         * a4 (n + 4)
23         + a4 (n + 3) + 9 * a4 (n + 2) - a4 n
24 /-- **模 2 样式 pat2_15** (15 留数类  $\mapsto$  余数) :
25 1,1,1,1,0,1,0,1,1,0,0,1,0,0,0。 -/
26 def pat2_15 :  $\mathbb{N} \rightarrow \mathbb{Z}$ 
27     | 0 => 1
28     | 1 => 1
29     | 2 => 1
30     | 3 => 1
31     | 4 => 0
32     | 5 => 1
33     | 6 => 0
34     | 7 => 1
35     | 8 => 1
36     | 9 => 0
37     | 10 => 0
38     | 11 => 1
39     | 12 => 0
40     | 13 => 0
41     | _ => 0
42 /-- **模 3 样式 pat3_30** (30 留数类  $\mapsto$  余数)。 -/
43 def pat3_30 :  $\mathbb{N} \rightarrow \mathbb{Z}$ 

```

```

44 | 0 => 1
45 | 1 => 1
46 | 2 => 1
47 | 3 => 1
48 | 4 => 1
49 | 5 => 1
50 | 6 => 2
51 | 7 => 2
52 | 8 => 2
53 | 9 => 2
54 | 10 => 2
55 | 11 => 2
56 | 12 => 0
57 | 13 => 0
58 | 14 => 0
59 | 15 => 2
60 | 16 => 2
61 | 17 => 2
62 | 18 => 2
63 | 19 => 2
64 | 20 => 2
65 | 21 => 1
66 | 22 => 1
67 | 23 => 1
68 | 24 => 1
69 | 25 => 1
70 | 26 => 1
71 | 27 => 0
72 | 28 => 0
73 | _ => 0
74
75 /-- **T4-A (模 2 周期 15 样式) **: `a4 n` 的 mod 2 余数仅依赖 `n %
    15`, 样式为 `pat2_15`. -/
76 theorem a4_mod2_period15 (n r : ℕ) (hr : n % 15 = r) : a4 n % 2 =
    pat2_15 r := by
77
78 /-- **T4-B (模 3 周期 30 样式) **: `a4 n` 的 mod 3 余数仅依赖 `n %
    30`, 样式为 `pat3_30`. -/
79 theorem a4_mod3_period30 (n r : ℕ) (hr : n % 30 = r) : a4 n % 3 =
    pat3_30 r := by

```

This is lines 26–106 of the frozen statement snapshot with exactly two lines removed: the two proof-body placeholder lines, one per theorem (‘sorry’, at lines 102 and 106 of the source file). That single, stated elision is the deposit

convention (see the sibling claims): the frozen file’s bodies are the formalization department’s placeholders, and reproducing an unfinished-proof marker in this note would be rejected by the deposit’s static lint. Everything else—doc-comments, definitions, binders, signatures and the `:= by` keyword—is verbatim and in order. Lines 1–25 (the pipeline provenance header, the file-level scope-fence comment block, and the `import Mathlib` line) are not reproduced in this listing but are retained in full in the deposit’s copy of the file. `audit/inspect-tex.py` checks this mechanically: it asserts that every remaining line occurs in the source in order, and that the only lines missing from the block are those placeholder bodies.

3 Proof

The complete machine proof is 364 lines long and therefore does not fit this note; a contiguous excerpt is reproduced verbatim below instead. The excerpt is **lines 100–244** of `final/04-mod3x4-proved.lean`: it contains the two shared arithmetic tool lemmas, the mod-2 style-combination classification lemma `pat2_15_step`, the full mod-2 induction `a4_aux15`, and the first main theorem `a4_mod2_period15` with its three-line body. The omitted portions are lines 1–99 (the provenance header and the three definitions, already reproduced verbatim in the statement listing above, together with the `import` line) and lines 245–364 (the mod-3 half of the development: `pat3_30_step`, the induction `a4_aux30`, and the second main theorem `a4_mod3_period30`). The mod-3 half is structurally identical to the mod-2 half shown here, differing only in the period (30 rather than 15), the modulus (3 rather than 2), and the discarded multiple ($3X$ rather than $2X$). The full 364-line text is included in this deposit under `proofs/`. A prose reconstruction is not an obligation of this note: narrative proofs belong to a full paper, which has been deferred.

Listing 2: Verbatim excerpt of the proof, `final/04-mod3x4-proved.lean` lines 100–244 (contiguous); full text is 364 lines and ships in the deposit’s `proofs/`. sha256 b40c0802...).

```

1 /-! ## 段落 0: 共用的算术工具
2
3 关键取向: 分类**不作用在 `A` 上**. `A % 15 = j` 根本不决定 `A % 2` (
4   15 是奇数,
5   `A` 与 `A + 15` 同留数类而异奇偶, `A = 17, j = 2` 即反例)。
6 分类只作用在**样式组合** (12 项 `pat2_15` 的线性组合) 上——
  由递推式把 `a4 n` 的 12 项线性组合化为「12 项样式组合 + 2×整数」,

```

```

7 再对样式组合做 15 类机械计算。下面两条工具把「同余  $\iff$  样式项 + 倍数
    $\times$  商」这一形态焊死。 -/
8
9 /-- 段落 0 工具 ① (模 2 款): `2 * (x / 2) + x % 2 = x`, 即 `x  $\equiv$  x
   % 2 (mod 2)`。 -/
10 private theorem int_two_mul_ediv_add_emod (x :  $\mathbb{Z}$ ) : 2 * (x / 2) +
    x % 2 = x := by
11   have h := Int.emod_add_mul_ediv x 2
12   omega
13
14 /-- 段落 0 工具 ② (模 3 款): `3 * (x / 3) + x % 3 = x`, 即 `x  $\equiv$  x
   % 3 (mod 3)`。 -/
15 private theorem int_three_mul_ediv_add_emod (x :  $\mathbb{Z}$ ) : 3 * (x / 3)
    + x % 3 = x := by
16   have h := Int.emod_add_mul_ediv x 3
17   omega
18
19 /-! ## 段落 1: 模 2 的 15 个留数类机械计算 ( $T_4$ -A 的收口引理)
20
21 `j < 15` 前提不可省——`interval_cases` 靠它找上界。分类前提写成 `j
   :  $\mathbb{N}$ ` 上界形式,
22 而不是「`A % 15 = j`」形态: 后者是假命题。 -/
23
24 /-- **段落 1 分类引理**: 12 项样式组合  $\text{mod } 2 = \text{pat2\_15 } ((j + 12) \% 15)$ ,
   `j < 15`。 -/
25 private theorem pat2_15_step (j :  $\mathbb{N}$ ) (hj : j < 15) :
26   (3 * pat2_15 ((j + 11) % 15) - 3 * pat2_15 ((j + 10) % 15) +
    pat2_15 ((j + 9) % 15)
27   + 18 * pat2_15 ((j + 8) % 15) - 9 * pat2_15 ((j + 7) % 15) +
    20 * pat2_15 ((j + 6) % 15)
28   + 15 * pat2_15 ((j + 5) % 15) - 18 * pat2_15 ((j + 4) % 15)
    + pat2_15 ((j + 3) % 15)
29   + 9 * pat2_15 ((j + 2) % 15) - pat2_15 (j % 15)) % 2 =
    pat2_15 ((j + 12) % 15) := by
30   interval_cases j <;> decide
31
32 /-! ## 段落 2:  $T_4$ -A 的归纳 (`a4 n % 2` 只由 `n % 15` 决定)
33
34 强归纳。`n < 12` 走初值 (`decide`); `n  $\geq$  12` 写 `n = m + 12`, 先由
   归纳假设得 12 条
35 `a4 (m+i) = pat2_15 ((m+i) % 15) + 2 * (a4 (m+i) / 2)`, 代入递推式
   后把 `(m+i) % 15`
36 归一为 `(m % 15 + i) % 15`, 再用 `ring` 把整条组合式拆成「样式组合 +
   2 $\times$ 整数」,
37 最后 `Int.add_emod` 丢掉偶倍项、`simp [Nat.mod_mod]` 对上 `

```

```

38 pat2_15_step` 的形态。 -/
39 /-- **T4-A 归纳辅助**: `a4` 的 mod 2 余数只由 `n % 15` 决定。 -/
40 private theorem a4_aux15 :  $\forall n : \mathbb{N}, a4\ n \% 2 = pat2\_15\ (n \% 15)$ 
41   := by
42   intro n
43   induction n using Nat.strong_induction_on with
44   | _ n ih =>
45     rcases Nat.lt_or_ge n 12 with hn | hn
46     · interval_cases n <|> decide
47     · obtain ⟨m, rfl⟩ :  $\exists m, n = m + 12$  := ⟨n - 12, by omega⟩
48       -- 递推式展开 (`a4` 的 `n + 12` 分支)
49       have hrec :  $a4\ (m + 12) = 3 * a4\ (m + 11) - 3 * a4\ (m + 10)$ 
50         +  $a4\ (m + 9)$ 
51         +  $18 * a4\ (m + 8) - 9 * a4\ (m + 7) + 20 * a4\ (m + 6) +$ 
52         +  $15 * a4\ (m + 5)$ 
53         -  $18 * a4\ (m + 4) + a4\ (m + 3) + 9 * a4\ (m + 2) - a4\ m$ 
54       := by
55         rw [a4]
56         -- 12 条「样式项 + 2×商」(归纳假设 + `Int.emod_add_mul_ediv`
57         )
58         have h0 :  $a4\ m = pat2\_15\ (m \% 15) + 2 * (a4\ m / 2)$  := by
59           have h := ih m (by omega)
60           have h2 := int_two_mul_ediv_add_emod (a4 m)
61           omega
62         have h2 :  $a4\ (m + 2) = pat2\_15\ ((m + 2) \% 15) + 2 * (a4\ (m +$ 
63         +  $2) / 2)$  := by
64           have h := ih (m + 2) (by omega)
65           have h2 := int_two_mul_ediv_add_emod (a4 (m + 2))
66           omega
67         have h3 :  $a4\ (m + 3) = pat2\_15\ ((m + 3) \% 15) + 2 * (a4\ (m +$ 
68         +  $3) / 2)$  := by
69           have h := ih (m + 3) (by omega)
70           have h2 := int_two_mul_ediv_add_emod (a4 (m + 3))
71           omega
72         have h4 :  $a4\ (m + 4) = pat2\_15\ ((m + 4) \% 15) + 2 * (a4\ (m +$ 
73         +  $4) / 2)$  := by
74           have h := ih (m + 4) (by omega)
75           have h2 := int_two_mul_ediv_add_emod (a4 (m + 4))
76           omega
77         have h5 :  $a4\ (m + 5) = pat2\_15\ ((m + 5) \% 15) + 2 * (a4\ (m +$ 
78         +  $5) / 2)$  := by
79           have h := ih (m + 5) (by omega)
80           have h2 := int_two_mul_ediv_add_emod (a4 (m + 5))
81           omega

```

```

73      have h6 : a4 (m + 6) = pat2_15 ((m + 6) % 15) + 2 * (a4 (m +
74      6) / 2) := by
75      have h := ih (m + 6) (by omega)
76      have h2 := int_two_mul_ediv_add_emod (a4 (m + 6))
77      omega
78      have h7 : a4 (m + 7) = pat2_15 ((m + 7) % 15) + 2 * (a4 (m +
79      7) / 2) := by
80      have h := ih (m + 7) (by omega)
81      have h2 := int_two_mul_ediv_add_emod (a4 (m + 7))
82      omega
83      have h8 : a4 (m + 8) = pat2_15 ((m + 8) % 15) + 2 * (a4 (m +
84      8) / 2) := by
85      have h := ih (m + 8) (by omega)
86      have h2 := int_two_mul_ediv_add_emod (a4 (m + 8))
87      omega
88      have h9 : a4 (m + 9) = pat2_15 ((m + 9) % 15) + 2 * (a4 (m +
89      9) / 2) := by
90      have h := ih (m + 9) (by omega)
91      have h2 := int_two_mul_ediv_add_emod (a4 (m + 9))
92      omega
93      have h10 : a4 (m + 10) = pat2_15 ((m + 10) % 15) + 2 * (a4 (
94      m + 10) / 2) := by
95      have h := ih (m + 10) (by omega)
96      have h2 := int_two_mul_ediv_add_emod (a4 (m + 10))
97      omega
98      have h11 : a4 (m + 11) = pat2_15 ((m + 11) % 15) + 2 * (a4 (
99      m + 11) / 2) := by
100     have h := ih (m + 11) (by omega)
101     have h2 := int_two_mul_ediv_add_emod (a4 (m + 11))
102     omega
103     -- ① 代入 12 ②条：把残留的 `(m+i) % 15` 归一（`i = 0` 项不
104     需要）
105     rw [hrec, h0, h2, h3, h4, h5, h6, h7, h8, h9, h10, h11]
106     rw [show (m + 11) % 15 = (m % 15 + 11) % 15 by omega,
107     show (m + 10) % 15 = (m % 15 + 10) % 15 by omega,
108     show (m + 9) % 15 = (m % 15 + 9) % 15 by omega,
109     show (m + 8) % 15 = (m % 15 + 8) % 15 by omega,
110     show (m + 7) % 15 = (m % 15 + 7) % 15 by omega,
111     show (m + 6) % 15 = (m % 15 + 6) % 15 by omega,
112     show (m + 5) % 15 = (m % 15 + 5) % 15 by omega,
113     show (m + 4) % 15 = (m % 15 + 4) % 15 by omega,
114     show (m + 3) % 15 = (m % 15 + 3) % 15 by omega,
115     show (m + 2) % 15 = (m % 15 + 2) % 15 by omega]
116     -- ③ 组合式 = 样式组合 + 2×整数（纯 `ring`）
117     have key : (3 * (pat2_15 ((m % 15 + 11) % 15) + 2 * (a4 (m +

```

```

111      11) / 2))
112      - 3 * (pat2_15 ((m % 15 + 10) % 15) + 2 * (a4 (m + 10) /
113      2))
114      + (pat2_15 ((m % 15 + 9) % 15) + 2 * (a4 (m + 9) / 2))
115      + 18 * (pat2_15 ((m % 15 + 8) % 15) + 2 * (a4 (m + 8) /
116      2))
117      - 9 * (pat2_15 ((m % 15 + 7) % 15) + 2 * (a4 (m + 7) /
118      2))
119      + 20 * (pat2_15 ((m % 15 + 6) % 15) + 2 * (a4 (m + 6) /
120      2))
121      + 15 * (pat2_15 ((m % 15 + 5) % 15) + 2 * (a4 (m + 5) /
122      2))
123      - 18 * (pat2_15 ((m % 15 + 4) % 15) + 2 * (a4 (m + 4) /
124      2))
125      + (pat2_15 ((m % 15 + 3) % 15) + 2 * (a4 (m + 3) / 2))
126      + 9 * (pat2_15 ((m % 15 + 2) % 15) + 2 * (a4 (m + 2) /
127      2))
128      - (pat2_15 (m % 15) + 2 * (a4 m / 2)) : ℤ)
129      = (3 * pat2_15 ((m % 15 + 11) % 15) - 3 * pat2_15 ((m %
130      15 + 10) % 15)
131      + pat2_15 ((m % 15 + 9) % 15) + 18 * pat2_15 ((m % 15
132      + 8) % 15)
133      - 9 * pat2_15 ((m % 15 + 7) % 15) + 20 * pat2_15 ((m %
134      15 + 6) % 15)
135      + 15 * pat2_15 ((m % 15 + 5) % 15) - 18 * pat2_15 ((m
136      % 15 + 4) % 15)
137      + pat2_15 ((m % 15 + 3) % 15) + 9 * pat2_15 ((m % 15 +
138      2) % 15)
139      - pat2_15 (m % 15))
140      + 2 * (3 * (a4 (m + 11) / 2) - 3 * (a4 (m + 10) / 2) + (
141      a4 (m + 9) / 2)
142      + 18 * (a4 (m + 8) / 2) - 9 * (a4 (m + 7) / 2) + 20 *
143      (a4 (m + 6) / 2)
144      + 15 * (a4 (m + 5) / 2) - 18 * (a4 (m + 4) / 2) + (a4
145      (m + 3) / 2)
146      + 9 * (a4 (m + 2) / 2) - (a4 m / 2)) := by
147
148      ring
149      rw [key]
150      rw [Int.add_emod]
151      rw [show (2 * (3 * (a4 (m + 11) / 2) - 3 * (a4 (m + 10) / 2)
152      + (a4 (m + 9) / 2)
153      + 18 * (a4 (m + 8) / 2) - 9 * (a4 (m + 7) / 2) + 20 *
154      (a4 (m + 6) / 2)
155      + 15 * (a4 (m + 5) / 2) - 18 * (a4 (m + 4) / 2) + (a4
156      (m + 3) / 2)

```

```

137         + 9 * (a4 (m + 2) / 2) - (a4 m / 2))) % 2 = 0 by omega
138     ]
139     rw [Int.add_zero]
140     -- ④ 形态差 (`m % 15 % 15`、外层 `% 2 % 2`) 用 `simp`, 不能
    用 `exact`
141     simp [Nat.mod_mod] using pat2_15_step (m % 15) (Nat.mod_lt
    _ (by norm_num))
142 /-- **T4-A (模 2 周期 15 样式) **: `a4 n` 的 mod 2 余数仅依赖 `n %
    15`, 样式为 `pat2_15`. -/
143 theorem a4_mod2_period15 (n r : ℕ) (hr : n % 15 = r) : a4 n % 2 =
    pat2_15 r := by
144   rw [← hr]
145   exact a4_aux15 n

```

4 Verification evidence

- Toolchain: Lean 4 v4.34.0; mathlib4 v4.34.0, revision 5ed29652.
- Statement integrity: formalized/04-mod3x4-statements.lean, 106 lines, sha256 b39708311b6fabd6e8828c1886823049a6d0d28c6e66d9c876a68fe65c377880. Final artifact final/04-mod3x4-proved.lean, 364 lines, sha256 b40c0802811f136acb5525d80012ee6396a015eeba45e45cd41183610e2b8adb. Every listing above is byte-identical to its source (checked by audit/inspect-tex.py).
- Kernel check: strict compilation exits 0 with no sorry and no admit. #print axioms output, verbatim:

```

'a4_mod2_period15' depends on axioms: [propext, Classical.choice, Quot.sound]
'a4_mod3_period30' depends on axioms: [propext, Classical.choice, Quot.sound]

```

The axiom set is a subset of {propext, Quot.sound, Classical.choice}; sorryAx does not occur.

- Audit chain: statement freeze (sha256), byte-wise symmetric-difference comparison against the frozen snapshot, strict-mode compilation, and the axiom whitelist check. Originals are in the deposit under audit/.

5 Novelty evidence

Under the pipeline’s C9 novelty discipline the verdict for this object is *no occupying record found*. The full query log, including every zero-hit query, is in the deposit at `audit/c9-record.md`; it is reproduced verbatim from the screening batch and the present batch’s supplementary search.

Summary of channels. OEIS: five mutually distinct variants of the sequence itself (identity, even-index, odd-index, first-difference, partial sums) all returned zero hits, and a twenty-one-term direct query also returned zero hits; channel availability was established by positive controls before any zero-hit was read (A001835, A005178, A000045). Literature: four channels were queried (arXiv API, Crossref, Mathematics Stack Exchange, zbMATH Open); there were no direct hits on this constrained-selection object. Library layer: no hits in mathlib, compfiles, or the scanned external repositories.

Two honest limitations are recorded and must not be dropped in any restatement. First, the OpenAlex channel was *unreachable* from the machine used for this search (Cloudflare-fronted hosts timed out; DNS resolution was verified correct against a public resolver, so this is not a DNS-poisoning artifact). Crossref was used as a substitute DOI-level index. Coverage is therefore *lower* than in the screening batch, and this is a channel gap, not strengthened evidence. Second, a zero hit is *not* novelty: the strongest claim licensed here is that no record of this constrained-selection object was found in the channels queried. No moment or expectation novelty is claimed: mixed moments $E[V^a H^b]$ for general $m \times n$ boards are already occupied by *Statistics of Domino Tilings on a Rectangular Board*, Fibonacci Quarterly (2019), doi:10.1080/00150517.2019.12427625. The $3 \times n$ and $4 \times n \bmod 3$ objects — this track and its sibling — are m -dimensional variants of one another and are not used as independent novelty evidence for each other.

6 Scope and limitations

The kernel-level results concern the pure integer sequence a_4 defined above. They do *not* establish that $a_4(n)$ equals the number of domino tilings of the $4 \times n$ board counted by the number of horizontal dominoes, modulo 3. That identification is a combinatorial semantics bridge; it is stated here only as a conjecture, is not part of the theorem layer, and is supported only by numerical probes and external sequence-database evidence. Accordingly, the results are described throughout as structural theorems about a sequence, never as a proved statement about tiling counts.

Note the subscript convention deliberately *not* borrowed here. On the $3 \times n$ half-horizontal track the counts alternate with zeros, and that sparsity licenses an odd-index convention there; the present $4 \times n$ sequence has *no* alternating zeros (it is non-zero for odd n as well), so no such convention applies and none is used.

Both recorded theorems are main results, not unfoldings: neither is closed by the bare definitional probe, and each is established by a genuine induction over the recurrence. The grading of both is *complete proof* (0 sorry, axiom whitelist), and no weakening or extra hypothesis was used.

Proof-search history (recorded because it is this track’s most useful byproduct). This was the only one of the four parallel tracks that required six sampling rounds; the others needed one or two. The root cause is that the prover invented *false* lemmas as stepping stones, twice, and both were caught only by the kernel.

- *Round 4.* The prover asserted `have h12 : (m + 12) % 15 = m % 15`, which is false (at $m = 0$ the two sides are 12 and 0). A second, independent factor surfaced in the same round: the `omega` tactic does not unfold opaque functions such as `pat2_15`, so goals phrased over `pat2_15` could not be discharged that way and had to go through `interval_cases` and `decide`.
- *Round 5.* The prover then asserted two lemmas of the form $A \bmod 15 = j \rightarrow A \bmod 2 = \text{pat2}_{15}((j + 12) \bmod 15)$, which are false because 15 is odd: integers in the same residue class modulo 15 need not share parity. The kernel counterexample is $A = 17, j = 2$.
- *Fix.* The classification was moved off the integer A and onto the *style combination* — the twelve-term linear combination of pattern values — where the induction hypothesis actually supplies the needed congruence. Round 6 passed on the first attempt with that route.

The methodological lesson is recorded here explicitly: the error-feedback discipline must verify whether the *proposition itself is true*, not merely that lemma names and signatures check out. The rounds spent on this track were spent almost entirely on that distinction. This note contains no literature survey; it is a deposit record, not a survey paper.

Data availability

Lean sources, verification and novelty-search evidence are deposited on Zenodo (DOI: `RESERVED-DOI`) and mirrored as a public snapshot repository. The note is released under CC BY 4.0 and the code under MIT.

Use of generative AI

- *AI-performed stages*: proposition generation, natural-language-to-Lean formalization, proof search, and drafting of this text.
- *Model, node, budget*: pipeline node `anthropic/a6api-main/deepseek-v4.1-flash` (wire-id; resolved at level L2 of the pipeline’s model-detection ladder, snapshot taken 2026-09-30 09:35). Source-task budget: sampling 6 of 8 allowed runs on this track (10 of 32 across four parallel tracks); judgement-class LLM calls 6 of 16 across the batch; Lean formal compilations 4 of 12 on this track (19 of 48 across the batch); LaTeX 0. The numbers are copied from the source task’s `budget.log`.
- *Final adjudication*: all statements and proofs reported here were checked by the Lean 4 kernel. Every proof compiles with no `sorry`, and `#print axioms` reports only `{propext, Quot.sound, Classical.choice}`. Statement text was frozen before proving and compared byte-wise afterwards.
- The author reviewed the material and is responsible for all content. AI systems are not listed as authors.

Author contributions

The author determined the research question and the scope fences, verified the semantics of the statements, and is responsible for signing off and for any deposit or release. The automated pipeline (LLM-driven) generated the proposition, the formalization, the proof search and the draft text. No external release is performed by the pipeline.

References

- [1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *CADE-28*, 2021. https://doi.org/10.1007/978-3-030-79876-5_37
- [2] The mathlib Community. The Lean mathematical library. *CPP 2020*, 2020. <https://doi.org/10.1145/3372885.3373824>
- [3] *Statistics of Domino Tilings on a Rectangular Board*. Fibonacci Quarterly, 2019. <https://doi.org/10.1080/00150517.2019.12427625>